บทที่ 6

ระบบนำทางและสถาปัตยกรรมการเปลี่ยนหน้าจอด้วย Expo Router

ในอดีต การจัดการระบบนำทาง (Navigation) ใน React Native ต้องอาศัยไลบรารี React Navigation ซึ่งมีความซับซ้อนในการตั้งค่าแผนผังหน้าจอ แต่สำหรับ **Expo Router** (เปิดตัวแบบสมบูรณ์ใน Expo SDK 54 / Router v6) ได้นำเสนอแนวคิดใหม่ที่เรียกว่า **File-based Routing** ซึ่งถอดแบบมาจากสถาปัตยกรรมระดับอุตสาหกรรมอย่าง Next.js บนฝั่งเว็บ ช่วยให้นักพัฒนาสร้างและบริหารเส้นทางเดินหน้าจอผ่านโครงสร้างโฟลเดอร์ได้อย่างสะดวก รวดเร็ว และลื่นไหล

6.1 แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์ (File-based Routing)

File-based Routing คือการแปลงตำแหน่งเส้นทางของไฟล์ (File Paths) ในโฟลเดอร์พิเศษที่ตั้งชื่อว่า **`app/`** ให้กลายมาเป็นเส้นทางการนำทาง (Route Paths) ของแอปพลิเคชันโดยอัตโนมัติ (Expo, 2024):

```
โครงสร้างในโฟลเดอร์ app/                URL / เส้นทางเข้าถึง
app/
├── index.tsx                          =>  "/" (หน้าแรกสุดของแอป)
├── profile.tsx                        =>  "/profile" (หน้าโปรไฟล์)
└── settings/
    └── index.tsx                      =>  "/settings" (หน้าตั้งค่า)
```

เมื่อคุณเพิ่มไฟล์ใหม่เข้าไปในโฟลเดอร์ `app/` ตัวระบบรวบรวมของ Expo (Metro Bundler) จะดักตรวจและเปิดใช้แผนที่นำทางนั้น ๆ ขึ้นมาโดยที่คุณไม่จำเป็นต้องมาเขียนโค้ดเพิ่มหน้าลงในอาเรย์การลงทะเบียนแต่อย่างใด

6.2 การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx

ไฟล์ที่ชื่อว่า **`_layout.tsx`** ถือเป็นชิ้นส่วนควบคุมพิเศษใน Expo Router (Expo, 2024) มีบทบาทสำคัญในการ:

1. กำหนดรูปแบบเลย์เอาต์หลักของแอป เช่น หน้าจอเรียงซ้อน (Stack) หรือ หน้าจอแบบแท็บบาร์ (Tabs)

2. ห่อหุ้มและแชร์ส่วนหน้าตาร่วมกัน (Shared UI) เช่น แถบสถานะ หรือแถบนำทางหลัก

3. จัดการพฤติกรรมความปลอดภัยและการปกป้องการเข้าถึงหน้าจอ (เช่น ระบบ Auth Guards)

ถ้าเราบันทึกไฟล์ **_layout.tsx** ไว้ที่ระดับบนสุดของโฟลเดอร์ **app/** สไตล์ของเลย์เอาต์นั้นจะคลุมทุกไฟล์ย่อยที่อยู่ภายใต้โฟลเดอร์นั้นทั้งหมด

6.3 ระบบนำทางแบบสะสมประวัติ (Stack Navigation)

เลย์เอาต์แบบ `Stack` คือระบบจัดการนำหน้าจอที่จำลองการวางซ้อนข้อมูลแผ่นการ์ดจากล่างขึ้นบน (Push/Pop Screen):

• เมื่อเปิดหน้าใหม่ หน้าจอใหม่จะถูกดันซ้อนทับอยู่ด้านบนสุดของหน้าเก่า (Push)

• เมื่อกดปุ่มย้อนกลับ (Back) หน้าบนสุดจะถูกดึงทิ้งออกไปเพื่อกลับมาโชว์หน้าจอด้านล่าง (Pop)

1. วิธีประกาศเลย์เอาต์ Stack ใน _layout.tsx

เราเรียกใช้คอมโพเนนต์ **<Stack>** เพื่อสร้างโครงสร้างการนำทางแบบซ้อนทับ:

```
// app/_layout.tsx
import { Stack } from 'expo-router';

export default function Layout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: '#0f172a' }, // แต่งสีแถบหัวข้อเข้ม
        headerTintColor: '#ffffff', // สีตัวหนังสือปุ่มย้อนกลับ
        headerTitleStyle: { fontWeight: 'bold' },
      }}
    >
      {/* ระบุตั้งค่าสไตล์เฉพาะหน้าจอผ่าน Stack.Screen */}
      <Stack.Screen name="index" options={{ title: 'หน้าหลักเดสก์ท็อป' }} />
      <Stack.Screen name="profile" options={{ title: 'โปรไฟล์ของคุณ' }} />
    </Stack>
  );
}
```

6.4 ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation)

เลย์เอาต์แบบ `Tabs` จัดสรรความสามารถยอดนิยมในการควบคุมเปลี่ยนโหมดหลักของแอปพลิเคชันผ่านเมนูปุ่มกดสัมผัสด้านล่างสุดของจอ:

1. วิธีสร้างเลย์เอาต์ Tabs ใน app/_layout.tsx

ใช้คำสั่งนำเข้าจาก expo-router และจัดวางไอคอนสไตล์:

```
// app/_layout.tsx
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons'; // โหลดไอคอนมาตรฐาน

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#38bdf8', // สีเมนูขณะถูกคลิกเลือก
        tabBarInactiveTintColor: '#94a3b8',
        tabBarStyle: { backgroundColor: '#0f172a' },
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: 'หน้าหลัก',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="home" color={color} size={size} />
    ),
        }}
   />
      <Tabs.Screen
        name="settings"
        options={{
          title: 'ตั้งค่า',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="settings" color={color} size={size} />
    ),
        }}
   />
    </Tabs>
 );
}
```

6.5 การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ (Link & Router Object)

ใน Expo Router มี 2 วิธีหลักในการสั่งนำทางเปลี่ยนหน้าจอ:

1. การนำทางแบบประกาศผ่านคอมโพเนนต์ <Link> (Declarative Navigation)

เหมาะอย่างยิ่งสำหรับข้อความ ตัวหนังสือ หรือปุ่มสัมผัสสเตติกทั่วไป โดยส่งพาธปลายทางผ่านพร็อพส์ href:

```
import { Link } from 'expo-router';
```

```
export default function HomeScreen() {
  return (
    <View>
      <Link href="/profile" asChild>
        {/* asChild สั่งให้ Link ไปคุมคุณสมบัติสัมผัสของคอมโพเนนต์ลูกย่อย */}
        <Pressable><Text>เข้าหน้าโปรไฟล์</Text></Pressable>
      </Link>
    </View>
  );
}
```

2. การสั่งการนำทางด้วยเหตุการณ์ JavaScript ผ่าน `router` (Imperative Navigation)

กรณีต้องการประมวลผลโลจิกสำเร็จก่อนนำทาง เช่น เช็คว่าผู้ใช้กรอกฟอร์มเสร็จแล้ว หรือดึงบันทึกสำเร็จ จึงค่อยย้ายหน้า:

- `router.push('/path')`: ผลักหน้าจอใหม่ทับซ้อนบนสแต็ค (ผู้ใช้ย้อนกลับได้)
- `router.replace('/path')`: ล้างหน้าจอปัจจุบันออกและสวมทับหน้าใหม่ (ใช้บ่อยในระบบ Auth ล็อกอินสำเร็จแล้วไม่ให้กดย้อนกลับมาหน้ารับค่าอีก)
- `router.back()`: ย้อนกลับหน้าจอก่อนหน้า 1 ระดับ

```
import { router } from 'expo-router';
import { Pressable, Text } from 'react-native';

export default function LoginButton() {
  const handleLogin = () => {
  // สมมติล็อกอินสำเร็จแล้ว
    router.replace('/dashboard'); // เปลี่ยนแบบแทนที่หน้าเดิมทันที
  };

  return (
    <Pressable onPress={handleLogin}>
      <Text>ยืนยันการเข้าสู่ระบบ</Text>
    </Pressable>
  );
}
```

6.6 เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์ (Dynamic Routes & useLocalSearchParams)

ในกรณีที่หน้าจอต้องการรับตัวแปร ID เพื่อไปยิงต่อค้นหารายละเอียดสินค้าเฉพาะชิ้น (เช่น หน้ารายละเอียดข่าว ID 125) เราใช้เทคนิคการตั้งชื่อไฟล์แบบใส่เครื่องหมายวงเล็บเหลี่ยม `[id].tsx` เพื่อกำหนดเป็น Dynamic Route

1. วิธีจัดเก็บโครงสร้างไฟล์ไดนามิก

```
app/
├── index.tsx                    (หน้ารวมข่าวสาร)
└── news/
    └── [id].tsx                 (หน้าอ่านข่าวเฉพาะชิ้น)
```

2. วิธีสั่งลิงก์นำทางส่งตัวแปร (Navigation with Parameters)

เราส่งพาธไดนามิกแนบข้อมูลพารามิเตอร์ไปได้โดยตรง:

```
// นำทางไปยังแอปหน้าอ่านข่าวรหัส 45
router.push('/news/45');

// หรือส่งผ่านพารามิเตอร์สไตล์ Query String เพิ่มเติม
router.push('/news/45?author=Somchai');
```

3. วิธีการดักจับค่าตัวแปรในหน้าปลายทางด้วย `useLocalSearchParams`

ภายในไฟล์ย่อย `app/news/[id].tsx` เราจะดึงข้อมูลตัวแปรจากพารามิเตอร์ทรานสิทผ่านมาด้วย `useLocalSearchParams` Hook:

```
// app/news/[id].tsx
import { useLocalSearchParams, router } from 'expo-router';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function NewsDetailScreen() {
 // ดักจับค่า id และพารามิเตอร์ author ที่ส่งผ่านเส้นทาง URL
  const { id, author } = useLocalSearchParams<{ id: string; author?: string }>();

  return (
    <View style={styles.container}>
      <Text style={styles.title}>รายละเอียดข่าวสารรหัส: #{id}</Text>
      {author && <Text style={styles.authorText}>ผู้เขียน: {author}</Text>}

      <Button title="กลับหน้าแรก" onPress={() => router.back()} />
    </View>
```

```
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  title: {
    fontSize: 20,
    color: '#ffffff',
    fontWeight: 'bold',
    marginBottom: 8,
  },
  authorText: {
    fontSize: 14,
    color: '#38bdf8',
    marginBottom: 20,
  },
});
```

สรุปท้ายบทเรียน

แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์: ระบบนำทางยุคใหม่ใน Expo SDK 54 ใช้สถาปัตยกรรมแบบ File-based Routing ผ่าน Expo Router v6 (Expo, 2024) โดยความสะดวกคือระบบจะแปลงพาธตำแหน่งโครงสร้างไฟล์ภายใต้โฟลเดอร์ app/ ให้กลายเป็นเส้นทางการเปลี่ยนหน้าตาแอปพลิเคชัน (URL Routes) โดยอัตโนมัติ ช่วยลดความยุ่งยากในการลงทะเบียนโค้ดแบบดั้งเดิม

การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx: ไฟล์ _layout.tsx เป็นกลไกศูนย์กลางเฉพาะของระบบมีประโยชน์ในด้านการควบคุมหน้าตาหลักของแอป เช่น หน้าจอ Stack หรือ หน้าจอแท็บบาร์ (Tabs) โดยทำหน้าที่เป็นโครงสร้างห่อหุ้มอินเทอร์เฟซส่วนกลาง (Shared UI) และช่วยจัดระเบียบความปลอดภัยในการกรองผู้เข้าใช้งานหน้าจอต่าง ๆ ได้อย่างเป็นระบบ

ระบบนำทางแบบสะสมประวัติ (Stack Navigation): โครงสร้าง Stack Navigation จำลองการนำทางในรูปแบบการวางแผ่นการ์ดซ้อนทับกัน (Push/Pop Screen) เมื่อเปิดหน้าจอใหม่จะนำหน้าต่างใหม่ไปทับแผ่นเก่า และเมื่อกดย้อนกลับหน้าจอด้านบนจะถูกดึงออกเพื่อกลับลงไป

แสดงแผ่นถัดลงไป โดยเราเรียกนำเข้าคอมโพเนนต์ `<Stack>` และปรับแต่งคุณสมบัติต่าง ๆ ในหน้า _layout.tsx

ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation): เลย์เอาต์แบบ Tabs เป็นการสร้างเมนูนำทางหลักสัมผัสด้านล่างสุดของสมาร์ทโฟนสำหรับสลับหมวดหลักของแอปพลิเคชันอย่างรวดเร็ว โดยสร้าง _layout.tsx แยกตามโฟลเดอร์ของกลุ่มงานและเรียกใช้คอมโพเนนต์ `<Tabs>` ร่วมกับ `<Tabs.Screen>` เพื่อระบุไอคอนและสไตล์สีเมื่อปุ่มทำงานหรือว่างเปล่าได้ตามที่ออกแบบไว้

การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ: การเปลี่ยนหน้าทำได้ 2 แนวทาง ได้แก่ การใช้คอมโพเนนต์ `<Link>` ร่วมกับพร็อพส์ href สำหรับการกดเชื่อมโยงทั่วไป และการใช้คำสั่ง JavaScript สั่งเปลี่ยนหน้าผ่านออบเจกต์ router (เช่น router.push สำหรับผลักหน้าจอขึ้นสแต็ค, router.replace เพื่อการข้ามหน้าแบบลบหน้าเดิมทิ้ง และ router.back สำหรับย้อนกลับหน้าเดิม)

เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์: สำหรับหน้าที่ต้องการรับตัวแปรระบุ ID เฉพาะ เราจะใช้การตั้งชื่อไฟล์แบบใส่เครื่องหมายวงเล็บเหลี่ยม เช่น [id].tsx เพื่อกำหนดเป็น Dynamic Route ซึ่งสามารถส่งผ่านพารามิเตอร์และตัวแปรผ่านเส้นทาง URL ได้อย่างอิสระ และฝั่งหน้าปลายทางจะดักจับข้อมูลเหล่านั้นมาใช้งานได้สะดวกผ่าน useLocalSearchParams Hook

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับสถาปัตยกรรม File-based Routing หน้าที่ของไฟล์ `_layout.tsx` เลย์เอาต์แบบ Stack และ Tabs วิธีเปลี่ยนหน้าจอด้วย `<Link>` และ `router` ตลอดจนการทำระบบจัดเส้นทางไดนามิกและการอ่านค่าด้วย `useLocalSearchParams` ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือคำอธิบายของแนวคิดระบบการจัดเส้นทางแบบ "File-based Routing" ใน Expo Router ได้ถูกต้องที่สุด?

ก. เส้นทางการเชื่อมต่อจะตั้งชื่อตามฟังก์ชันภายในของ Database

ข. โครงสร้างไฟล์และโฟลเดอร์ภายในโฟลเดอร์ `app/` จะถูกนำมาใช้ระบุขอบเขตเส้นทางการนำหน้าจอของแอปพลิเคชันโดยอัตโนมัติ

ค. ต้องทำรายการประกาศเชื่อมพาธในไฟล์ CSS เสมอ

ง. แอปพลิเคชันจะแปลงหน้าทั้งหมดให้ออกมารันบนเว็บบราวเซอร์ผ่านชื่อไฟล์เท่านั้น

ข้อ 2: โฟลเดอร์ใดที่เป็นตำแหน่งรากหลัก (Root Directory) สำหรับการตั้งวางและตรวจจับไฟล์เส้นทางหน้าจอของระบบ Expo Router?

ก. `src/`

ข. `components/`

ค. `app/`

ง. `assets/`

ข้อ 3: ไฟล์พิเศษที่ใช้ชื่อว่า `_layout.tsx` มีหน้าที่และความสำคัญหลักอย่างไรในโฟลเดอร์เส้นทางนำทาง?

ก. คอยจัดเตรียมสไตล์สีและฟอนต์ภายนอกทั้งหมดโดยไม่เกี่ยวกับหน้าจอ

ข. ใช้ระบุกรุปเลย์เอาต์การนำทางร่วมกัน (เช่น Stack หรือ Tabs) ให้แก่กลุ่มไฟล์ในโฟลเดอร์ระดับเดียวกันและโฟลเดอร์ย่อย

ค. เป็นคลังเก็บไฟล์รูปภาพไอคอนของโปรเจกต์

ง. ล้างค่าย้อนหลังข้อมูลสเตตเมื่อผู้ใช้งานปิดเปิดเครื่องใหม่

ข้อ 4: ในระบบนำทางแบบ Stack Navigation พฤติกรรมแบบดั้งเดิมในการจัดการสแต็คหน้าจอเป็นอย่างไร?

ก. หน้าจอจะสลับเรียงแนวนอนแบบข้างชนข้างโดยไม่จัดหน้าทับซ้อนกัน

ข. แสดงผลหน้าจอใหม่ซ้อนทับขึ้นมาอยู่ด้านบนสุด (Push) และดึงหน้าจอนั้นออกไปเพื่อกลับสู่หน้าด้านล่าง (Pop)

ค. นำทางผู้ใช้ไปยังเบราว์เซอร์ภายนอกเพื่อดาวน์โหลดรูปภาพ

ง. ปิดหน้าจออื่นทั้งหมดทันทีและเหลือเพียงหน้าแรกสุดเสมอ

ข้อ 5: หากต้องการแก้ไขชื่อหัวข้อแถบส่วนหัว (Header Title) สำหรับหน้าจอเฉพาะเจาะจงที่อยู่ภายใต้เลย์เอาต์ `<Stack>` ควรเขียนอย่างไร?

ก. กำหนดใน Property ชื่อ `title` ของสไตล์ชีท

ข. เรียกใช้งานคอมโพเนนต์ `<Stack.Screen options={{ title: 'ชื่อหัวข้อที่ต้องการ' }} />` ภายในไฟล์หน้าจอนั้น

ค. ตั้งชื่อไฟล์ด้วยอักขระตัวพิมพ์ใหญ่ทั้งหมด

ง. ส่งค่าตัวหนังสือไปป้อนที่ตัวแปรของ `useLocalSearchParams`

ข้อ 6: การระบุพร็อพส์ `asChild` ในคอมโพเนนต์ `<Link>` (เช่น `<Link href="/profile" asChild>`) ส่งผลอย่างไรต่อคอมโพเนนต์ลูกด้านใน?

ก. สั่งให้ล้างประวัติการนำทางทั้งหมดทิ้งและจำลองเป็นหน้าแรกสุด

ข. ทำหน้าที่คอยจำลองการทำงานส่งต่อเหตุการณ์กดสัมผัส (onPress) และคุณสมบัติการนำทางไปควบคุมชิ้นส่วนของคอมโพเนนต์ลูกแทนการสร้างแท็ก `<a>` ครอบ

ค. บังคับเปลี่ยนรูปภาพลูกให้ยืดหยุ่นตามขนาดของขอบจอ

ง. สั่งเปิดลิงก์ไปยังเว็บเบราว์เซอร์ภายนอก

ข้อ 7: การนำทางโดยเรียกใช้งานคำสั่ง `router.replace('/home')` แตกต่างจากการเรียกใช้งาน `router.push('/home')` อย่างไร?

ก. `replace` จะผลักหน้าซ้อนสแต็คต่อประวัติเดิม แต่ `push` จะล้างข้อมูลออก

ข. `replace` จะทำลายหน้าจอปัจจุบันและสวมทับเส้นทางใหม่ลงไปแทนที่ประวัติเดิม (ย้อนกลับมาไม่ได้) ส่วน `push` จะวางหน้าจอทับซ้อนเพิ่มขึ้นไปในสแต็คสะสม

ค. `replace` ทำงานได้เฉพาะระบบ iOS ส่วน `push` ทำงานเฉพาะระบบ Android

ง. `replace` ไม่รองรับการส่งผ่านค่าข้อมูลไดนามิกพารามิเตอร์

ข้อ 8: หากต้องการออกแบบหน้าต่างแสดงรายละเอียดไอเท็มแบบรับตัวแปรไดนามิก ID เช่น พาธ `/details/12` หรือ `/details/abc` ผู้พัฒนาต้องตั้งชื่อไฟล์อย่างไร?

ก. `details-id.tsx`

ข. `[details].tsx`

ค. `[id].tsx` เก็บไว้ในโฟลเดอร์ชื่อ `details`

ง. `details.id.tsx`

ข้อ 9: ในหน้าจอปลายทางรับค่าเส้นทางไดนามิก เราจะดึงข้อมูลตัวแปรพารามิเตอร์ที่พ่วงส่งผ่านมาดักรับใช้งานด้วยเครื่องมือ (Hook) ตัวใด?

ก. `useState`

ข. `useLocalSearchParams`

ค. `useRouteOptions`

ง. `useNavigationState`

ข้อ 10: เลย์เอาต์การนำทางประเภทใดที่เหมาะสำหรับการสร้างแถบเมนูปุ่มนำทางหลักยึดติดอยู่ท้ายสุดของหน้าจอ เพื่อสลับหน้าการทำงานใหญ่ ๆ ในแอป?

ก. `<Stack>`

ข. `<Tabs>`

ค. `<Drawer>`

ง. `<ScrollView>`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การนำทางในแอปมือถือไม่เกี่ยวกับการจัดการดาต้าเบสหลังบ้านโดยตรง

• **ข. ถูก:** File-based Routing ของ Expo Router ทำงานโดยจับคู่พิกัดไฟล์ใน **app/** มาแปลเป็น Route นำทางอัตโนมัติ ช่วยลดความซับซ้อนในการเขียนแผนผังลงทะเบียน

• **ค. และ ง. ผิด:** CSS จัดตกแต่งสีสันเท่านั้น และแอปมือถือรันระบบนำทางระดับเนทีฟไม่ใช่รันเว็บพาธตรงตัว

เฉลย ข้อ 2: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** เป็นโฟลเดอร์เก็บซอร์สโค้ดอื่น คอมโพเนนต์แชร์ หรือภาพประกอบที่ไม่ใช่จุดตั้งเส้นทางนำทางหลัก

• **ค. ถูก:** Expo Router บังคับออกแบบให้โฟลเดอร์ **app/** ที่ระดับรากโครงการเป็นตำแหน่งจับคู่เส้นทางนำทางทั้งหมด

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ดีไซน์ภายนอกฟอนต์จะใช้การโหลด Asset ปกติ

• **ข. ถูก:** ไฟล์ **`_layout.tsx`** ใช้คุมเส้นทางร่วมกัน หากเราประกาศ Stack หรือ Tabs ไว้ในนี้ หน้าจอย่อยทั้งหมดในชั้นโฟลเดอร์เดียวกันจะทำงานอยู่ใต้สไตล์เลย์เอาต์นำทางดังกล่าว

• **ค. และ ง. ผิด:** ไม่ใช่ที่รวมภาพ และสเตตจะล้างตามปกติหากปิดตัวแอป

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การสลับเรียงแนวนอนแบบ Slide เป็นเพียง Transition ไม่ใช่หลักพฤติกรรมการจัดการสแต็ค

• **ข. ถูก:** โครงสร้างนำทางแบบ Stack ทำงานแบบ LIFO (Last In First Out) บล็อกใหม่ซ้อนบนสุด (Push) และเคลียร์ออกด้านหลังเพื่อกลับจุดเดิม (Pop)

• **ค. และ ง. ผิด:** ไม่ใช่คำสั่งนำออกเบราว์เซอร์ และยังมีกลไกคงหน้าเก่าค้างไว้ใต้สแต็ค

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สไตล์ชีทไม่สามารถเปลี่ยนข้อความของแถบหัวเรื่องระบบเนทีฟได้

• **ข. ถูก:** ตัวเสริม `<Stack.Screen>` ของ Expo Router เปิดช่องให้เราสั่ง Override ค่าตัวเลือก (Options) ประจำหน้าจอได้จากไฟล์ของหน้านั้น ๆ โดยตรง

• **ค. และ ง. ผิด:** การตั้งชื่อไฟล์ด้วยตัวพิมพ์ใหญ่ไม่ส่งผลต่อชื่อหัวข้อ และพารามิเตอร์มีไว้รับส่งตัวแปรข้อมูล

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวกับการเคลียร์สแต็ค ปรับขนาดรูป หรือเชื่อมเว็บภายนอก

• **ข. ถูก:** พร็อพส์ `asChild` เป็นตัวช่วยส่งต่อ Event Handler `onPress` ของปุ่มลิงก์ผ่านลงไปยังปุ่ม `Pressable` หรือชิ้นงานลูกย่อยด้านใน ป้องกันการซ้อนทับกันของแท็กระบบนำทาง

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สลับพฤติกรรมกัน

• **ข. ถูก:** `router.push()` จะวางหน้าจอทับซ้อนเพิ่มในประวัติทำให้กดย้อนกลับได้ ส่วน `router.replace()` จะเปลี่ยนหน้าแบบทับแทนที่หน้าเก่าทันที เหมาะสำหรับหน้าระบบลงทะเบียนเสร็จแล้วไม่อยากให้ย้อนกลับมาเจอหน้าฟอร์มกรอกซ้ำ

• **ค. และ ง. ผิด:** ทั้งคู่ทำงานได้เหมือนกันทั้งสองระบบ OS และสนับสนุนการส่งพารามิเตอร์ปกติ

เฉลย ข้อ 8: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** ชื่อรูปแบบอื่นไม่ถูกแปลงเป็นตัวแปรไดนามิกในระบบ Expo Router v6

• **ค. ถูก:** การห่อชื่อไฟล์ด้วยสัญลักษณ์วงเล็บเหลี่ยม `[id].tsx` เป็นการระบุว่า คีย์ชื่อ `id` จะเป็นตัวดักจับสตริงข้อมูลใน URL หรือพาธเพื่อนำไปประมวลผลต่อ

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `useState` จัดการสถานะภายในของตัวหน้านั้น ไม่เกี่ยวข้องกับการแปลงตัวแปร URL

• **ข. ถูก:** `useLocalSearchParams` ถูกสร้างขึ้นโดยเฉพาะใน Expo Router เพื่ออ่านค่าพารามิเตอร์ที่แนบมากับเส้นทางเดินหน้าจอปลายทาง (เช่น ดึงค่า ID สินค้า)

• **ค. และ ง. ผิด:** ไม่ใช่ชื่อคำสั่งมาตรฐาน

เฉลย ข้อ 10: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: Stack เหมาะสำหรับหน้าจอย่อยซ้อนลึก Drawer เหมาะสำหรับแถบปัดด้านข้าง ส่วน ScrollView ใช้สำหรับเลื่อนข้อความไม่ใช่เลย์เอาต์การเดินหน้าจอหลัก

• ข. ถูก: เลย์เอาต์แบบ `<Tabs>` ออกแบบมาจัดวางแท็บเดินหน้าระดับล่าง (Bottom Tab Bar) เพื่อความลื่นไหลในการสลับใช้งานฟีเจอร์หลัก ๆ ของแอป

ปฏิบัติการท้ายบทเรียน: การสร้างระบบนำทางหน้าข้อมูลจุดท่องเที่ยวแบบไดนามิก

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานระบบจัดเส้นทางเดินหน้าจอด้วยโครงสร้างโฟลเดอร์และไฟล์ของ **Expo Router** โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **แนะแนวแหล่งท่องเที่ยวออนไลน์ (Travel Guide & Destinations Directory)** ซึ่งเชื่อมโยงการทำงานแบบหน้าจอหลักเชื่อมสู่หน้ารายละเอียดแบบส่งผ่านค่าตัวแปรไดนามิกพารามิเตอร์ข้ามหน้าจอ

1. วัตถุประสงค์การเรียนรู้

1. สามารถออกแบบและตั้งค่าสถาปัตยกรรมระบบนำทางแบบซ้อนทับ (Stack) ด้วยไฟล์ `_layout.tsx`
2. สามารถเขียนฟังก์ชันนำทางไปยังจุดหมายต่าง ๆ โดยเรียกใช้ออบเจกต์คำสั่ง `router`
3. สามารถกำหนดโครงสร้างแฟ้มงานในรูปแบบไดนามิกลิงก์ (`app/destination/[id].tsx`) เพื่อดักจับรับค่าพารามิเตอร์
4. สามารถเรียกใช้ Hook `useLocalSearchParams` เพื่อรับข้อมูลรหัสประจำเมืองและนำมาจัดทำเนื้อหาเฉพาะตัวสำเร็จ
5. สามารถอัปเดตสไตล์และข้อความแถบส่วนหัวบาร์ (Dynamic Header Options) ผ่านคอมโพเนนต์ `<Stack.Screen>`

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนโค้ดจัดโครงสร้างระบบสตรีมนำทางภายใต้โฟลเดอร์หลัก `app/` ดังนี้:

โครงสร้างโฟลเดอร์ปฏิบัติการ:

```
app/
├── _layout.tsx                    => เลย์เอาต์ Stack ควบคุมหลัก
├── index.tsx                      => หน้าหลักรวมการ์ดเมืองท่องเที่ยว (Home List Screen)
└── destination/
    └── [id].tsx                   => หน้ารายละเอียดท่องเที่ยวเจาะลึกเฉพาะเมือง (Dynamic details)
```

รายละเอียดและเงื่อนไขการนำทาง:

1. **เลย์เอาต์หลัก (`app/_layout.tsx`):**
   - กำหนดให้เป็นระบบ `<Stack>`
   - ตกแต่งบาร์ด้านบนให้เป็นเฉดสี Slate โทนน้ำเงินเข้ม (`#1e293b`) ตัวหนังสือสีขาว
   - เปิดระบบแสดงผลปุ่มย้อนกลับสีฟ้าสว่าง (`#38bdf8`) อัตโนมัติบนหน้าย่อย
2. **หน้าหลักรวมจุดท่องเที่ยว (`app/index.tsx`):**
   - มีคลังข้อมูลเมืองท่องเที่ยวเป็นอาร์เรย์ (เช่น Tokyo, Paris, New York) ประกอบด้วย ID ชื่อประเทศ และชื่อภาพ
   - แสดงรูปภาพเมืองและป้ายสั้นโดยจัดด้วย `FlatList`
   - เมื่อผู้เรียนกดคลิกสัมผัสการ์ดเมืองใด ๆ จะต้องทริกเกอร์เรียกคำสั่ง `router.push()` เพื่อนำทางไปยังหน้ารายละเอียดเมืองนั้น พร้อมระบุพ่วงส่ง ID และพารามิเตอร์ชื่อเมือง (Query string) ไปด้วย
3. **หน้ารายละเอียดจุดท่องเที่ยวแบบไดนามิก (`app/destination/[id].tsx`):**
   - ดักจับรหัส ID และตัวแปร Query จากเส้นทางด้วย Hook `useLocalSearchParams`
   - เปลี่ยนข้อความหัวข้อบาร์ด้านบนสุด (Header Title) ของหน้าจอให้กลายเป็น "ข้อมูลของเมือง: [ชื่อเมือง]" ไดนามิกอัตโนมัติด้วย `<Stack.Screen options={{ title: ... }} />`
   - แสดงรายละเอียดคำบรรยายและภาพเมืองนั้น ๆ
   - มีปุ่ม "ย้อนกลับหน้าหลัก" สไตล์โมเดิร์นคุมด้วย `Pressable` ที่จะกระตุ้นคำสั่ง `router.back()` ย้อนหน้ากลับ

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ตั้งค่าเลย์เอาต์นำทาง Stack หลักใน `app/_layout.tsx`

สร้างหรือแก้ไขไฟล์ `app/_layout.tsx` เพื่อนำเสนอรูปแบบแผนผังการจัดหน้าซ้อนสะสม (Stack):

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: {
          backgroundColor: '#0f172a', // ธีมหัวข้อสีเข้ม
```

```
        },
        headerTintColor: '#38bdf8',    // ปุ่มกด Back สีฟ้านีออน
        headerTitleStyle: {
          fontWeight: 'bold',
          color: '#ffffff',            // ตัวหนังสือสีขาว
        },
        contentStyle: {
          backgroundColor: '#0f172a', // คุมสีพื้นหลังหน้าจอทั้งหมดใน Stack
      }
        }}
      >
        {/* หน้าแรกจะแสดงหัวข้อ "แหล่งท่องเที่ยวทั่วโลก" */}
        <Stack.Screen
          name="index"
          options={{ title: 'แหล่งท่องเที่ยวทั่วโลก' }}
    />
        {/* ส่วนหน้าไดนามิกจะตั้งค่า title แบบยืดหยุ่นในไฟล์ตัวเอง */}
       <Stack.Screen
          name="destination/[id]"
          options={{ title: 'กำลังโหลดข้อมูลเมือง...' }}
    />
      </Stack>
  );
}
```

ขั้นตอนที่ 2: สร้างหน้าจอลิสต์เมืองท่องเที่ยวส่งตัวแปรไดนามิกใน app/index.tsx

ใช้คำสั่งนำเข้าจาก expo-router และเขียนจำลองปุ่มส่งตัวแปรลิงก์:

```
import { View, Text, FlatList, Pressable, Image, StyleSheet } from 'react-
native';
import { router } from 'expo-router';

// ชุดข้อมูลจำลองจุดท่องเที่ยว
const DESTINATIONS = [
  { id: '1', name: 'โตเกียว (Tokyo)', country: 'ประเทศญี่ปุ่น', image:
'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=300' },
  { id: '2', name: 'ปารีส (Paris)', country: 'ประเทศฝรั่งเศส', image:
'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=300' },
  { id: '3', name: 'นิวยอร์ก (New York)', country: 'สหรัฐอเมริกา', image:
'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=300' },
];

export default function HomeList() {
  return (
    <View style={styles.container}>
      <FlatList
        data={DESTINATIONS}
```

```
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
      // สั่งนำทางแบบ Push ส่งค่า ID ไปที่ Path และพ่วงชื่อเมืองเป็น Query String
            onPress={() =>
router.push(`/destination/${item.id}?cityName=${item.name}`)}
         style={({ pressed }) => [
              styles.card,
              { opacity: pressed ? 0.95 : 1 }
      ]}
          >
            <Image source={{ uri: item.image }} style={styles.image} />
            <View style={styles.info}>
        <Text style={styles.title}>{item.name}</Text>
              <Text style={styles.subtitle}>{item.country}</Text>
            </View>
          </Pressable>
    )}
        contentContainerStyle={styles.list}
   />
    </View>
 );
}

const styles = StyleSheet.create({
 // รายละเอียดการจัดสไตล์คาร์ดจะรวมอยู่ในโค้ดเฉลยเต็ม
});
```

ขั้นตอนที่ 3: รับค่าพารามิเตอร์และแสดงหน้าจอเฉพาะเมืองใน app/destination/[id].tsx

ดึงพารามิเตอร์มาจำลองข้อมูลเนื้อหาและการเซ็ตค่า Header อัตโนมัติ:

```
import { useLocalSearchParams, router, Stack } from 'expo-router';
import { View, Text, Image, Pressable, StyleSheet } from 'react-native';

// คำบรรยายข้อมูลท่องเที่ยวจำลองแยกตาม ID
const DESTINATION_DETAILS: Record<string, { desc: string; rating: string;
cover: string }> = {
  '1': {
    desc: 'โตเกียว เมืองหลวงของญี่ปุ่น ผสมผสานความทันสมัยจากตึกระฟ้าและวัฒนธรรมโบราณจากวัดและศาลเจ้าได้
อย่างลงตัว มีแหล่งช้อปปิ้งและอาหารเลิศรสรอคุณอยู่',
    rating: '☆ 4.9 / 5',
    cover: 'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=500'
  },
  '2': {
    desc: 'ปารีส เมืองหลวงแห่งแฟชั่นและศิลปะของฝรั่งเศส เต็มไปด้วยสถาปัตยกรรมสุดคลาสสิก หอไอเฟล พิพิธภัณฑ์ลูฟร์
และบรรยากาศสุดโรแมนติกริมแม่น้ำแซน',
```

```
    rating: '☆ 4.8 / 5',
    cover: 'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=500'
  },
  '3': {
    desc: 'นิวยอร์ก นครที่ไม่เคยหลับใหลของสหรัฐฯ แหล่งรวมความบันเทิงระดับโลก ไทม์สแควร์ เซ็นทรัลพาร์ก โรงละครบรอดเวย์ และความหลากหลายทางวัฒนธรรม',
    rating: '☆ 4.7 / 5',
    cover: 'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=500'
  },
};

export default function DestinationDetail() {
 // ดึกรับพารามิเตอร์ ID และตัวแปร Query String
  const { id, cityName } = useLocalSearchParams<{ id: string; cityName?: string }>();

  const detail = DESTINATION_DETAILS[id];

  if (!detail) {
    return (
      <View style={styles.centerContainer}>
        <Text style={styles.errorText}>ไม่พบข้อมูลเมืองท่องเที่ยวนี้</Text>
      </View>
  );
  }

  return (
    <View style={styles.container}>
      {/* ตั้งค่า Title บาร์ด้านบนตามชื่อเมืองที่รับมาแบบเรียลไทม์ */}
      <Stack.Screen options={{ title: cityName || 'รายละเอียดเมือง' }} />

      <Image source={{ uri: detail.cover }} style={styles.coverImage} />

      <View style={styles.content}>
        <Text style={styles.title}>{cityName}</Text>
        <Text style={styles.rating}>{detail.rating} คะแนนยอดนิยม</Text>
        <Text style={styles.desc}>{detail.desc}</Text>

        {/* ปุ่มกดย้อนกลับหน้าแรกจัดด้วย Pressable */}
        <Pressable
          onPress={() => router.back()}
          style={({ pressed }) => [
            styles.backButton,
            pressed ? styles.backButtonPressed : null
    ]}
        >
          <Text style={styles.backButtonText}>ย้อนกลับหน้าหลัก</Text>
        </Pressable>
```

```
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  // การจัดระยะสเปซจะแสดงในส่วนของเฉลยสมบูรณ์
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำข้อมูลชุดโค้ดเฉลยเต็มรูปแบบด้านล่างนี้แยกบันทึกตามเส้นทางไฟล์ของโปรเจกต์ เพื่อทดสอบการเปลี่ยนหน้าจอส่งผ่านพารามิเตอร์ที่สวยงามแบบเนทีฟ:

4.1 ซอร์สโค้ดไฟล์เลย์เอาต์: app/_layout.tsx

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: {
          backgroundColor: '#0f172a',
        },
        headerTintColor: '#38bdf8',
        headerTitleStyle: {
          fontWeight: 'bold',
          color: '#ffffff',
        },
        contentStyle: {
          backgroundColor: '#0f172a',
        }
      }}
    >
      <Stack.Screen
        name="index"
        options={{ title: 'จุดหมายปลายทางยอดนิยม' }}
    />
      <Stack.Screen
        name="destination/[id]"
        options={{ title: 'รายละเอียดสถานที่ท่องเที่ยว' }}
    />
    </Stack>
  );
}
```

4.2 ซอร์สโค้ดหน้าหลักสารบัญเมือง: app/index.tsx

```
import { View, Text, FlatList, Pressable, Image, StyleSheet } from 'react-
native';
import { router } from 'expo-router';

const DESTINATIONS = [
  { id: '1', name: 'โตเกียว (Tokyo)', country: 'ประเทศญี่ปุ่น', image:
'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=300' },
  { id: '2', name: 'ปารีส (Paris)', country: 'ประเทศฝรั่งเศส', image:
'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=300' },
  { id: '3', name: 'นิวยอร์ก (New York)', country: 'สหรัฐอเมริกา', image:
'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=300' },
];

export default function HomeList() {
  return (
    <View style={styles.container}>
      <Text style={styles.listHeader}>สำรวจเมืองในฝันของคุณ</Text>

      <FlatList
        data={DESTINATIONS}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
            onPress={() =>
router.push(`/destination/${item.id}?cityName=${item.name}`)}
            style={({ pressed }) => [
       styles.card,
              { transform: [{ scale: pressed ? 0.98 : 1.0 }] }
      ]}
          >
            <Image source={{ uri: item.image }} style={styles.image} />
            <View style={styles.info}>
              <Text style={styles.title}>{item.name}</Text>
              <Text style={styles.subtitle}>{item.country}</Text>
            </View>
            <Text style={styles.arrowIcon}>chevron-right ➔</Text>
          </Pressable>
    )}
        contentContainerStyle={styles.list}
   />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
```

```
    paddingHorizontal: 16,
  },
  listHeader: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#94a3b8',
    marginVertical: 16,
  },
  list: {
    paddingBottom: 20,
  },
  card: {
    flexDirection: 'row',
    backgroundColor: '#1e293b',
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 16,
    padding: 12,
    alignItems: 'center',
  },
  image: {
    width: 70,
    height: 70,
    borderRadius: 12,
    backgroundColor: '#334155',
  },
  info: {
    flex: 1,
    marginLeft: 16,
  },
  title: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  subtitle: {
    fontSize: 13,
    color: '#94a3b8',
    marginTop: 4,
  },
  arrowIcon: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: 'bold',
    marginRight: 4,
  },
});
```

4.3 ซอร์สโค้ดหน้าแสดงผลละเอียดท่องเที่ยว: app/destination/[id].tsx

```
import { useLocalSearchParams, router, Stack } from 'expo-router';
import { View, Text, Image, Pressable, StyleSheet, ScrollView } from
'react-native';

const DESTINATION_DETAILS: Record<string, { desc: string; rating: string;
cover: string }> = {
  '1': {
    desc: 'โตเกียว เมืองหลวงของญี่ปุ่น ผสมผสานความทันสมัยจากตึกระฟ้าและวัฒนธรรมโบราณจากวัดและศาลเจ้าได้
อย่างลงตัว มีแหล่งช้อปปิ้งและอาหารเลิศรสรอคุณอยู่ ไม่ว่าจะเป็นการเดินชมย่านชิบูย่า พักผ่อนที่สวนอูเอโนะ หรือสัมผัสอาหาร
รสชาติแท้ๆ ในย่านชินจูกุ',
    rating: '☆ 4.9 / 5',
    cover: 'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=600'
  },
  '2': {
    desc: 'ปารีส เมืองหลวงแห่งแฟชั่นและศิลปะของฝรั่งเศส เต็มไปด้วยสถาปัตยกรรมสุดคลาสสิก หอไอเฟล พิพิธภัณฑ์ลูฟร์
และบรรยากาศสุดโรแมนติกริมแม่น้ำแซน การเดินชมสตรีทคาเฟ่ที่มีกลิ่นอายย้อนยุคริมทาง ชิมครัวซองต์อบสดใหม่ และชมแสงสี
วิจิตรในยามค่ำคืน',
    rating: '☆ 4.8 / 5',
    cover: 'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=600'
  },
  '3': {
    desc: 'นิวยอร์ก นครที่ไม่เคยหลับใหลของสหรัฐฯ แหล่งรวมความบันเทิงระดับโลก ไทม์สแควร์ เซ็นทรัลพาร์ก โรงละคร
บรอดเวย์ และความหลากหลายทางวัฒนธรรม ตลอดจนตึกเอ็มไพร์สเตตที่ตั้งตระหง่านท้าทายทุกสายตาผู้มาเยือนจากทั่วสารทิศ',
    rating: '☆ 4.7 / 5',
    cover: 'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=600'
  },
};

export default function DestinationDetail() {
  const { id, cityName } = useLocalSearchParams<{ id: string; cityName?:
string }>();

  const detail = DESTINATION_DETAILS[id];

  if (!detail) {
    return (
      <View style={styles.centerContainer}>
        <Text style={styles.errorText}>ไม่พบข้อมูลเมืองท่องเที่ยวนี้</Text>
        <Pressable onPress={() => router.back()}
style={styles.backButton}>
          <Text style={styles.backButtonText}>ย้อนกลับหน้าแรก</Text>
        </Pressable>
      </View>
  );
```

```
  }

  return (
    <ScrollView style={styles.container}>
      {/* ตั้งค่าหัวข้อหน้าจอแบบไดนามิกอิงตามชื่อเมือง */}
      <Stack.Screen options={{ title: cityName || 'รายละเอียดสถานที่' }} />

     <Image source={{ uri: detail.cover }} style={styles.coverImage} />

      <View style={styles.content}>
        <Text style={styles.title}>{cityName}</Text>
        <View style={styles.ratingBadge}>
          <Text style={styles.ratingText}>{detail.rating}</Text>
        </View>

        <Text style={styles.sectionTitle}>เกี่ยวกับสถานที่นี้</Text>
        <Text style={styles.desc}>{detail.desc}</Text>

        <Pressable
          onPress={() => router.back()}
          style={({ pressed }) => [
            styles.backButton,
            pressed ? styles.backButtonPressed : null
    ]}
        >
          <Text style={styles.backButtonText}>ย้อนกลับหน้าหลัก</Text>
        </Pressable>
      </View>
    </ScrollView>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  centerContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
    padding: 20,
  },
  coverImage: {
    width: '100%',
    height: 250,
    backgroundColor: '#334155',
  },
```

```
  content: {
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  ratingBadge: {
    backgroundColor: '#1e293b',
    borderWidth: 1,
    borderColor: '#334155',
    alignSelf: 'flex-start',
    paddingVertical: 4,
    paddingHorizontal: 10,
    borderRadius: 8,
    marginTop: 8,
    marginBottom: 20,
  },
  ratingText: {
    color: '#fbbf24',
    fontSize: 13,
    fontWeight: 'bold',
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#38bdf8',
    marginBottom: 8,
  },
  desc: {
    fontSize: 15,
    color: '#cbd5e1',
    lineHeight: 24,
    marginBottom: 28,
  },
  backButton: {
    backgroundColor: '#38bdf8',
    height: 48,
    borderRadius: 12,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 10,
    marginBottom: 20,
  },
  backButtonPressed: {
    backgroundColor: '#0ea5e9',
```

```
  },
  backButtonText: {
    color: '#0f172a',
    fontSize: 16,
    fontWeight: 'bold',
  },
  errorText: {
    color: '#f43f5e',
    fontSize: 16,
    marginBottom: 20,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Create Routes with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/create-pages/

Expo. (2024). *Expo Router Introduction*. จาก https://docs.expo.dev/versions/v54.0.0/router/introduction/

Expo. (2024). *Layouts with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/layouts/